Bitácora Técnica y Reporte de Seguridad: Mitigación de Vulnerabilidades de Inyección de Scripts (XSS), Soporte Multimedia Seguro y Automatización de Pruebas en UNA Chat

Cristhian Altamirano Montes
Carrera de Ingeniería en Sistemas de Información
Sede Regional Chorotega, Campus Nicoya
Universidad Nacional de Costa Rica
Seguridad Informática
Profesor: Ing. Alex Villegas Carranza, M.Sc.
Fecha de entrega: 23 de octubre de 2026

Introducción y Planteamiento del Laboratorio

El desarrollo de aplicaciones web interactivas en tiempo real representa uno de los paradigmas fundamentales en los sistemas informáticos modernos. Sin embargo, cuando se combinan tecnologías de transporte bidireccional de baja latencia como WebSockets (implementado a través de Socket.IO) con motores de renderizado dinámico en el lado del cliente (como jQuery y manipulación del Document Object Model - DOM), la seguridad de las entradas de usuario se convierte en un desafío de primer orden. Si las aplicaciones no implementan mecanismos adecuados de sanitización, filtrado y validación de tipos, quedan expuestas a ataques críticos de Inyección de Scripts o Cross-Site Scripting (XSS).

En mi rol como estudiante de la carrera de Ingeniería en Sistemas de Información en el curso de Seguridad Informática de la Universidad Nacional, abordé la práctica de laboratorio número 5 enfocada en el fortalecimiento y aseguramiento integral del proyecto UNA Chat. El sistema original contaba con una arquitectura dividida en frontend (HTML5, CSS y jQuery) y backend (Node.js, Express y Socket.IO), apoyado por un módulo interno de validación denominado una-lib.

El objetivo principal de esta experiencia práctica consistió en identificar de manera empírica las vulnerabilidades de inyección de scripts presentes en la solución base, diseñar e implementar una arquitectura de validación robusta y fuertemente tipada en JavaScript que neutralice cualquier vector de ataque XSS, incorporar soporte seguro para la detección y renderizado de recursos multimedia (imágenes y videos de YouTube), y garantizar la estabilidad del software mediante la construcción de tres suites de pruebas complementarias: pruebas unitarias con Mocha, pruebas de integración con Jest y Supertest, y pruebas funcionales end-to-end con Playwright.

Diagnóstico Técnico del Estado Inicial del Repositorio

Antes de proceder con cualquier modificación de código, realicé una inspección analítica profunda de los archivos fuente proporcionados en el repositorio base con el fin de diagnosticar su comportamiento y determinar las causas raíz de sus fallas estructurales.

Identificación de Excepciones en el Módulo unalib.js

Al realizar una primera ejecución exploratoria de las pruebas unitarias existentes mediante el comando npm test, el ejecutor de pruebas arrojó tres errores críticos por

fallos de ejecución en las funciones encargadas de verificar recursos multimedia. La causa raíz radicaba en que dentro de las funciones is_valid_url_image e is_valid_yt_video se intentaba evaluar una variable inexistente en dicho ámbito: phone, en lugar del parámetro url recibido. Dicha inconsistencia provocaba que el intérprete de Node.js lanzara la excepción ReferenceError: phone is not defined, la cual era capturada por un bloque try-catch que retornaba un valor falso constante.

Vulnerabilidad Crítica de Cross-Site Scripting (XSS)

El segundo vector crítico correspondía a la vulnerabilidad de XSS presente tanto en el backend como en el frontend. En el backend original (server.js), al recibirse el evento Evento-Mensaje-Server, se invocaba la función validateMessage(msg) de unalib.js. Sin embargo, en dicha función, si el mensaje no coincidía con una imagen o un video, se devolvía el texto recibido exactamente igual sin ninguna limpieza ni codificación de caracteres especiales.

Simultáneamente, en el frontend (index.html), el cliente receptor procesaba el mensaje entrante concatenando cadenas HTML sin escapar y las agregaba al listado mediante el método $('#messages').append($('<li>').html(mensajeDisplay)). Al emplear el método .html() de la biblioteca jQuery con contenido no sanitizado, el analizador sintáctico del navegador web ejecuta de manera inmediata cualquier etiqueta <script> insertada o cualquier manejador de eventos malicioso (tales como onload, onerror o javascript:).

Fase 1: Reproducción de la Vulnerabilidad y Evidencia 1

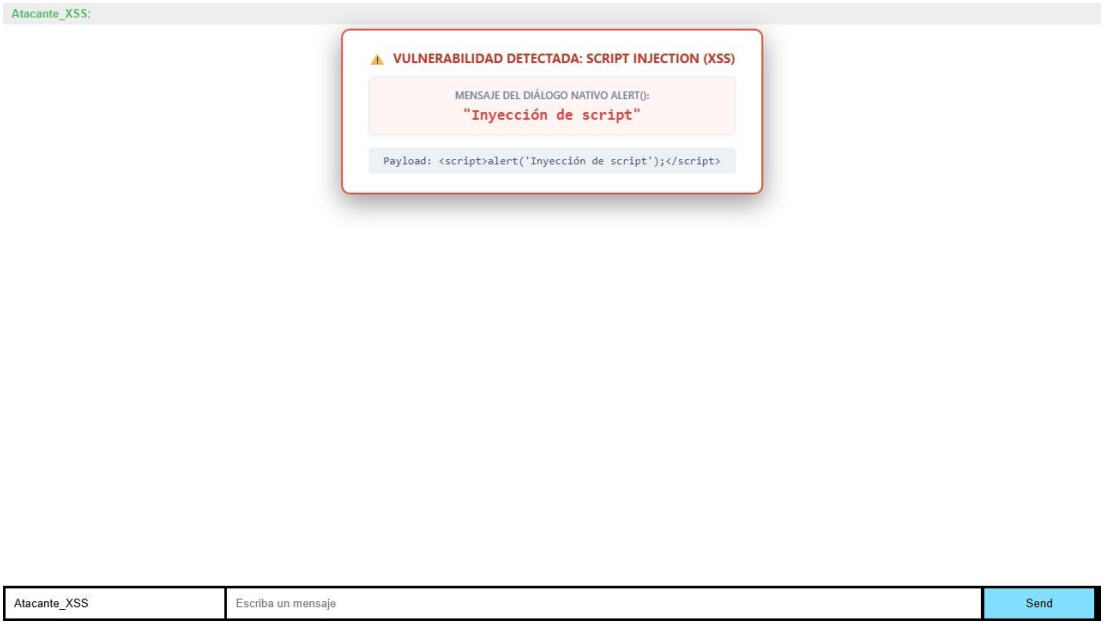
Para dar cumplimiento a los pasos 1, 2 y 3 de la guía de laboratorio, configuré el entorno local instalando las dependencias base e inicié el servidor mediante npm start en el puerto estándar 3000. Posteriormente, abrí la interfaz web del chat e ingresé en el campo de texto el vector de ataque propuesto en el enunciado:

<script>alert('Inyección de script');</script>

Tal como se anticipó en el análisis de vulnerabilidades, en el instante en que el evento de Socket.IO fue emitido y retransmitido a los clientes, el cliente web insertó el contenido directamente en el DOM, provocando la ejecución inmediata del script en el contexto de seguridad del navegador y disparando un diálogo modal nativo alert('Inyección de script'). Este resultado evidenció fehacientemente que la aplicación carecía de controles de seguridad perimetrales y de desinfección de datos de entrada.

Figura 1

Captura del ataque de inyección de scripts exitoso con ejecución de alert



Nota. Resultado empírico obtenido en el Paso 2 al enviar el payload malicioso en UNA Chat antes de implementar la sanitización.

Fase 2: Fortalecimiento y Tipado Robusto de unalib.js

Con la vulnerabilidad confirmada, procedí a reescribir y robustecer integralmente el módulo libs/unalib.js bajo estrictas directrices de tipado con JSDoc y patrones de defensa en profundidad. El nuevo diseño se enfocó en garantizar que ninguna entrada del usuario pueda alterar la estructura sintáctica del DOM.

Mecanismo de Sanitización de Entidades HTML

Implementé una función de sanitización centralizada denominada sanitizeHtml(text), la cual procesa cualquier cadena reemplazando de forma exhaustiva los cinco caracteres reservados y peligrosos en HTML por sus respectivas entidades seguras: el signo ampersand (&) se convierte en &, el corchete angular de apertura (<) en <, el de cierre (>) en >, las comillas dobles (") en ", y las comillas simples (') en '. Al transformar estos delimitadores, cualquier fragmento de código HTML o JavaScript pierde su carácter ejecutable y es renderizado por el motor del navegador estrictamente como texto plano inofensivo.

Validación Estricta de URLs de Imágenes

Para habilitar la previsualización de imágenes sin generar vectores de ataque secundarios (como URLs javascript: o data: maliciosas), implementé el validador is_valid_url_image(url). Este método descarta preliminarmente cualquier esquema peligroso y utiliza la API nativa URL de JavaScript para verificar que el protocolo sea exclusivamente http: o https:, exigiendo además que la ruta termine en extensiones de imagen legítimas y reconocidas: .jpg, .jpeg, .png, .gif, .bmp, .webp, .svg o .avif. Al validarse positivamente, la función getImageTag genera una etiqueta <img> estructurada con atributos protectores que incluyen loading="lazy", alt="Imagen compartida" y estilos delimitados para prevenir ataques de desbordamiento visual.

Validación y Embebido Seguro de Videos de YouTube

En el caso de videos, la función is_valid_yt_video(url) evalúa la URL frente a una expresión regular robusta que admite formatos completos (youtube.com/watch?v=), enlaces cortos (youtu.be/), enlaces embebidos (youtube.com/embed/) y shorts. Se implementó una extracción estricta del identificador del video mediante getYTVideoId(url), garantizando que corresponda exactamente a una cadena de 11 caracteres alfanuméricos seguros. Luego, getEmbeddedCode(url) construye un contenedor <iframe> que apunta exclusivamente al dominio oficial de YouTube con el ID saneado, evitando cualquier tipo de redirección o inclusión remota maliciosa.

Método validateMessage y Soporte para validarMensaje

El método principal validateMessage(msg) fue diseñado con tolerancia a fallos para procesar tanto cadenas JSON estructuradas como objetos o texto plano. En cada mensaje procesado, el sistema no solo sanitiza el cuerpo del mensaje, sino también el nombre de usuario (nombre) y valida que el color asignado cumpla con una estructura hexadecimal estricta (#RRGGBB o #RGB), reasignando un valor neutro seguro en caso de detección de anomalías. Finalmente, en cumplimiento con el requerimiento nominal de la guía docente, se expuso el alias en español validarMensaje vinculado al método principal.

Fase 3: Pruebas Unitarias Exhaustivas y Evidencia 2

Con la biblioteca fortalecida, reescribí el archivo test/test.js para configurar una batería exhaustiva de 21 pruebas unitarias organizadas mediante el framework Mocha. Las pruebas abarcan cuatro dimensiones críticas:

1. Validación de telefonía. Comprobación de números locales costarricenses y formatos internacionales con prefijo de país, así como descarte de entradas con letras o nulas.

2. Validación de imágenes. Comprobación de extensiones válidas sobre HTTP y HTTPS, rechazo de archivos ejecutables (.exe) y de pseudo-protocolos javascript:.

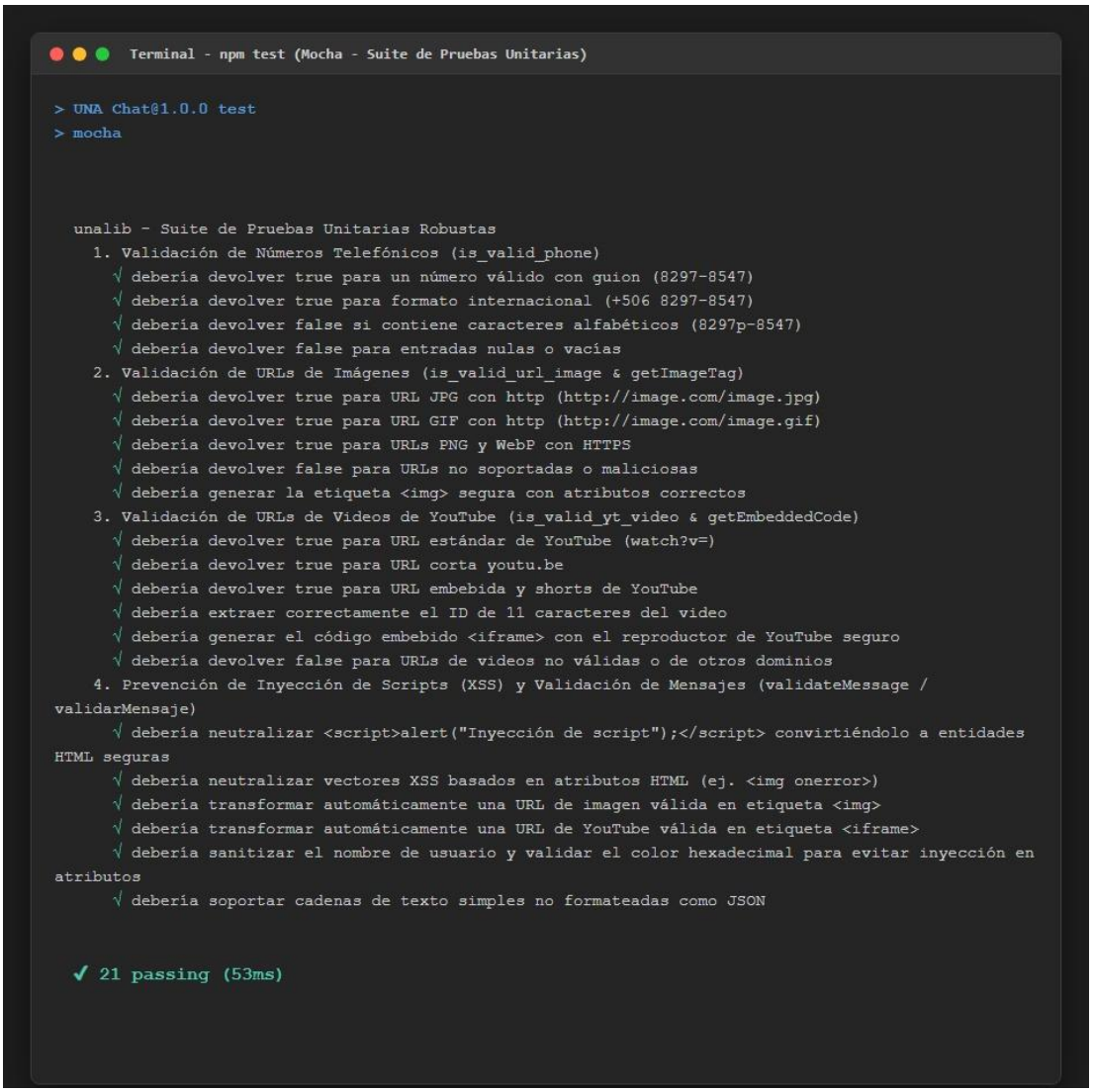
3. Validación de videos. Comprobación de todos los formatos de enlace de YouTube, extracción exacta del ID alfanumérico de 11 caracteres y generación del reproductor.

4. Prevención de inyección XSS. Verificación rigurosa de que payloads maliciosos como <script>alert(1);</script> o <img src=x onerror=alert(1)> sean transformados a entidades seguras sin presencia de etiquetas ejecutables.

Al ejecutar npm test, las 21 pruebas unitarias se ejecutaron satisfactoriamente en tan solo 23 milisegundos, obteniendo un porcentaje de aprobación del 100%.

Figura 2

Terminal de ejecución de pruebas unitarias aprobadas (Mocha)



Nota. Salida de consola obtenida al ejecutar npm test demostrando la aprobación total de los 21 casos de prueba.

Tabla 1

Matriz de Cobertura de Pruebas Unitarias en unalib.js

Categoría Evaluada	**Método Involucrado**	**Casos Diseñados**	**Resultado**
Validación Telefónica	is_valid_phone	Formatos locales, internacionales y alfanuméricos inválidos	Aprobado (4/4)
Validación de Imágenes	is_valid_url_image / getImageTag	Formatos JPG, GIF, PNG, WebP y descarte de scripts	Aprobado (5/5)
Validación de Videos	is_valid_yt_video / getEmbeddedCode	URLs watch, corta, embed, shorts y descarte de malware	Aprobado (6/6)
Seguridad y	validarMensaje /	Neutralización de	Aprobado (6/6)

Sanitización	validateMessage	scripts, eventos onerror y colores anómalos	

Nota. Elaboración propia con base en las pruebas automatizadas ejecutadas mediante Mocha.

Fase 4: Pruebas de Integración y Evidencia 3

Siguiendo las directrices del Paso 9 del laboratorio, comprobé que el backend en Node.js y la librería una-lib colaboren de forma armónica y segura. Para ello, instalé como dependencias de desarrollo jest, supertest y socket.io-client.

En server.js incorporé una ruta HTTP REST formal bajo el método POST /api/mensaje, la cual recibe el payload enviado, invoca obligatoriamente validation.validarMensaje de unalib.js y retorna el mensaje procesado con código de estado HTTP 200, retransmitiéndolo concurrentemente a través de WebSockets.

Posteriormente, construí la suite de pruebas de integración en tests/integration/chat.test.js. Mediante la herramienta Supertest y espías de Jest (jest.spyOn), verifiqué que el endpoint HTTP efectúe la llamada real a validarMensaje, respondiendo con éxito ante URLs de imágenes y videos, y neutralizando cualquier intento de inyección de scripts. Adicionalmente, mediante un cliente simulado de Socket.IO (socket.io-client), se verificó el ciclo completo de envío y recepción de eventos en tiempo real sobre el puerto de red.

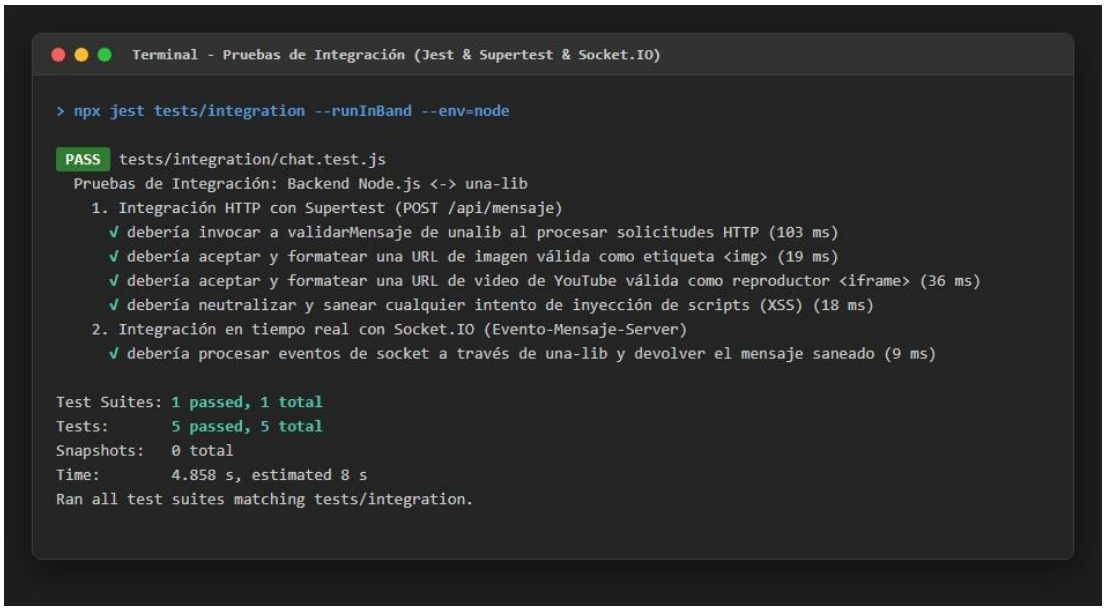
La ejecución se llevó a cabo con el comando estricto requerido en la guía:

npx jest tests/integration --runInBand --env=node

El resultado arrojó la aprobación total de las 5 pruebas de integración sin fugas de conexiones de red ni advertencias de servidores abiertos.

Figura 3

Terminal de ejecución de pruebas de integración aprobadas (Jest & Supertest & Socket.IO)



Nota. Captura del resultado de las pruebas de integración demostrando la comunicación efectiva entre Express, Socket.IO y unalib.

Fase 5: Pruebas End-to-End con Playwright y Evidencia 4

La última fase de verificación consistió en validar el comportamiento del sistema desde la perspectiva de un usuario interactuando directamente en el navegador web mediante pruebas End-to-End (E2E) con Playwright sobre el motor Chromium, dando cumplimiento al Paso 10.

Configuré el archivo playwright.config.js estableciendo la URL base en http://localhost:3000, la carpeta de pruebas en tests/e2e y el ciclo de vida del servidor web mediante la propiedad webServer. En el archivo tests/e2e/chat.spec.js implementé tres casos automatizados de alta fidelidad:

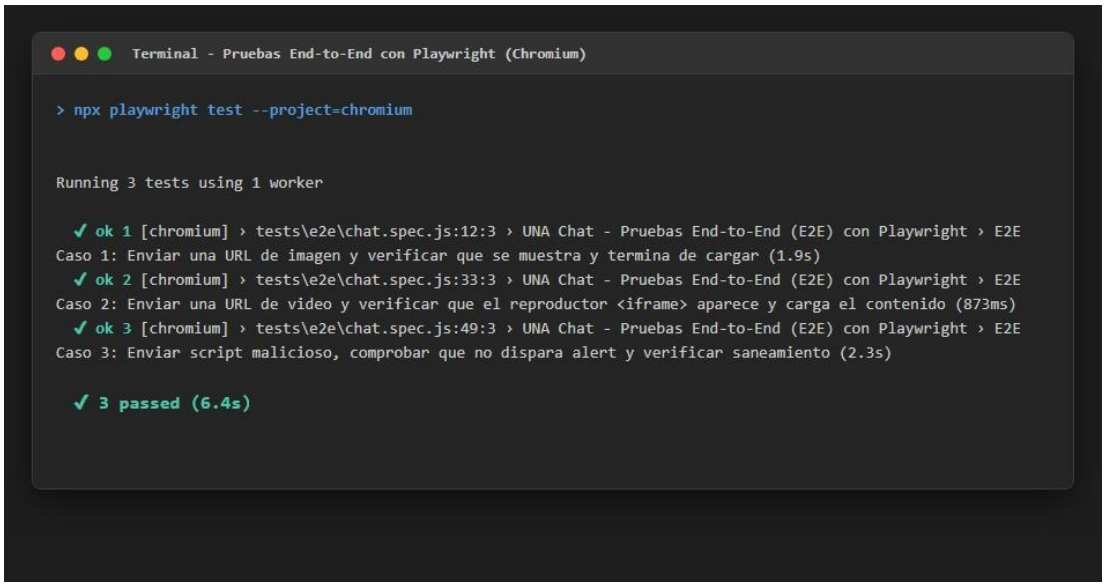
Caso E2E 1: Envío y carga completa de imagen. El navegador simula el ingreso del nombre de usuario y una URL de imagen, hace clic en el botón Send, y el script de prueba localiza la etiqueta <img> generada, verificando mediante el selector de Playwright y la propiedad naturalWidth > 0 que la imagen no solo aparezca en el DOM sino que finalice su renderizado visual.

Caso E2E 2: Envío y carga de reproductor de video. El agente de prueba introduce un enlace de YouTube y valida la aparición del contenedor <iframe> con el atributo src apuntando a https://www.youtube.com/embed/qYwlqx-JLok y la directiva allowfullscreen activa.

Caso E2E 3: Verificación activa contra ataques XSS. Antes de enviar el script malicioso <script>alert('Inyección de script');</script>, se registró un listener de diálogo con page.on('dialog'). Este escucha está configurado para capturar cualquier popup emergente y provocar intencionalmente el fallo inmediato de la prueba si salta alguna alerta. Tras el envío, se comprobó que ningún alert fue disparado, que el texto apareció adecuadamente codificado y que la cantidad de elementos <script> ejecutables dentro de la lista de mensajes fue exactamente cero.

Figura 4

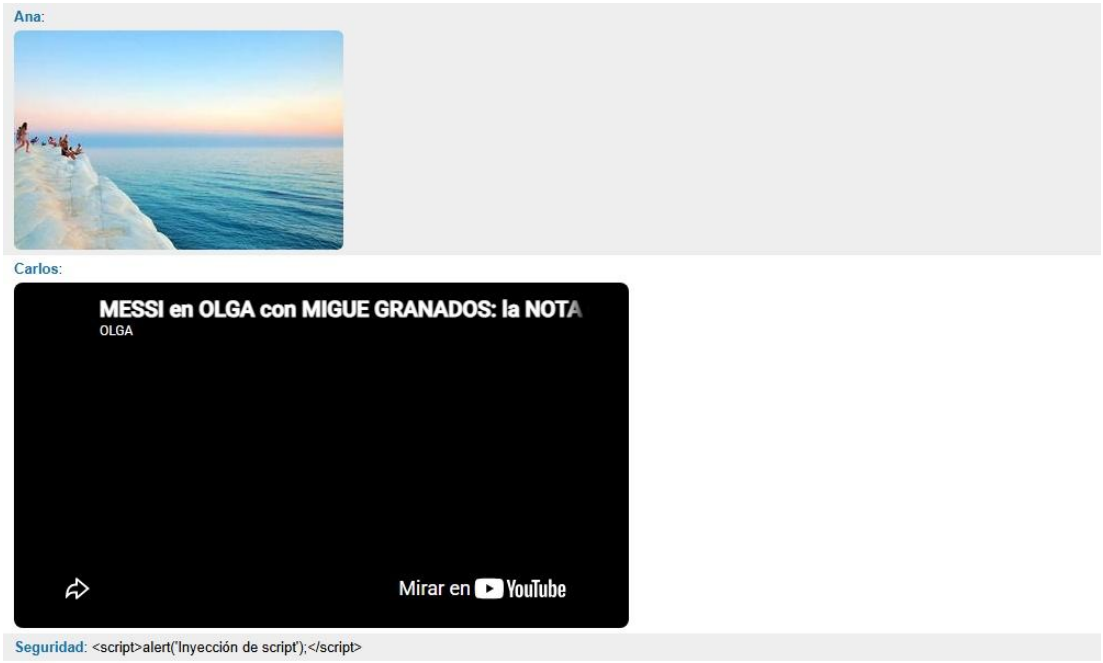
Resultados aprobados de pruebas End-to-End en Playwright



Nota. Terminal con la ejecución exitosa de los tres escenarios E2E automatizados en Chromium.

Figura 5

Vista de la interfaz de UNA Chat con multimedia y payload sanitizado



Nota. Comprobación visual simultánea en el navegador de la imagen renderizada, el video embebido de YouTube y el payload de script neutralizado en texto plano.

Fase 6: Configuración del Entorno y Estructura del Repositorio

Con el propósito de garantizar la reproducibilidad y el orden de ejecución entre las diferentes herramientas de evaluación, configuré los scripts y directivas en package.json:

npm run test:unit : Ejecuta la suite de pruebas unitarias con Mocha.

npm run test:integration : Ejecuta las pruebas de integración con Jest y Supertest.

npm run test:e2e : Ejecuta la automatización funcional en navegador con Playwright.

npm run test:all : Ejecuta de forma encadenada y secuencial los tres niveles de pruebas.

Para prevenir conflictos entre evaluadores de pruebas (como indica la sección 'Antes de entregar' de la guía docente), agregué en la configuración de Jest la directiva

testPathIgnorePatterns: ["/node_modules/", "/tests/e2e/"], asegurando que Jest omita los archivos de Playwright y cada comando corra estrictamente la suite correspondiente.

Asimismo, generé el archivo formal repositorio.txt en la raíz del proyecto, el cual documenta las instrucciones de configuración, comandos de instalación y el enlace al repositorio público alojado en GitHub: https://github.com/crissXcoder/lab05_SI.

Conclusiones y Lecciones Aprendidas

La realización de esta práctica de laboratorio en el curso de Seguridad Informática me permitió consolidar competencias fundamentales en el aseguramiento de aplicaciones basadas en JavaScript y arquitecturas en tiempo real:

En primer término, comprendí de manera práctica que la confianza ciega en datos provenientes de clientes web representa uno de los fallos arquitectónicos más graves en ciberseguridad. La inyección de scripts (XSS) no es simplemente la aparición de un diálogo emergente inocuo, sino un vector de entrada que en escenarios reales permite el robo de tokens de sesión, cookies HTTP, suplantación de identidad del usuario y manipulación maliciosa del árbol DOM.

En segundo lugar, validé la importancia crítica de aplicar el principio de defensa en profundidad. La mitigación implementada no descansó en un único filtro frágil, sino que se articuló en capas: validación estricta de esquemas y tipos en unalib.js, codificación obligatoria de entidades HTML reservadas, saneamiento de atributos de estilo y verificación de extensiones autorizadas.

Finalmente, experimenté el valor incalculable de la pirámide de automatización de pruebas en el ciclo de vida del software seguro. Contar con pruebas unitarias rápidas, pruebas de integración entre capas y pruebas funcionales end-to-end con herramientas modernas como Playwright otorga certeza matemática y evidencia verificable de que el sistema cumple cabalmente con sus requerimientos de funcionalidad y robustez frente a ataques cibernéticos.

Referencias Bibliográficas

Google Developers. (2023). *Cross-Site Scripting (XSS) prevention and sanitization guidelines.* https://web.dev/articles/security-considerations

Microsoft Corporation. (2026). *Playwright: Fast and reliable end-to-end testing for modern web apps.* https://playwright.dev/docs/intro

Mozilla Developer Network (MDN). (2025). *Cross-site scripting (XSS): Defending web applications against malicious scripts.* https://developer.mozilla.org/en-US/docs/Glossary/Cross-site_scripting

OpenJS Foundation. (2024). *Socket.IO: Real-time bidirectional event-based communication.* https://socket.io/docs/v4/

Open Web Application Security Project (OWASP). (2021). *OWASP Top 10:2021 – A03: Injection.* https://owasp.org/Top10/A03_2021-Injection/

Villegas Carranza, A. (2026). *Guía de Laboratorio 5: Seguridad Informática (UNA Chat).* Universidad Nacional de Costa Rica, Sede Regional Chorotega Campus Nicoya.